Example: SUBSET-SUM

Last week we saw a few examples of $\mathcal{NP}$-complete problems:3COL, CUBIC-SUBGRAPH, CLIQUE, INDEPENDENT-SET, and VERTEX-COVER. But these are all graph problems. What do we do if we're looking at a very different sort of problem?

Let's define SUBSET-SUM as:

> **Input:** A multiset S of non-negative integers, an integer t.
>
> **Question:** Is there a subset $S' \subseteq S$ such that $\sum_{x \in S'} = t$?

1. Proof that Subset-Sum is in $\mathcal{NP}$:
    1.
    2.
    3.
    4.
    5.

Example: SUBSET-SUM (2)

Remember that to prove 3SAT $\leqslant_p$ SUBSET-SUM, we want to encode a 3SAT instance into a SUBSET-SUM instance.

Given a 3SAT instance ϕ, we'll try to follow three basic steps:

- Find a way to code SUBSET-SUM to reflect the truth assignments for ϕ.
- Extend the resulting instance to reflect the resulting truth values in the clauses.
- Add any fudge factors we need to balance everything out.

Example: SUBSET-SUM (3)

So, given an input 3CNF ϕ, we'll write the variables of ϕ as $x_1, \ldots, x_\ell$ and its clauses as $c_1, \ldots, c_k$.

Idea: Let's start by setting up a table like so:

	1	2	3	...	ℓ	
x_1	1	0	0	...	0	$\langle$ something $\rangle$
$\overline{x_1}$	1	0	0	...	0	$\langle$ something $\rangle$
x_2	0	1	0	...	0	$\langle$ something $\rangle$
$\overline{x_2}$	0	1	0	...	0	$\langle$ something $\rangle$
x_3	0	0	1	...	0	$\langle$ something $\rangle$
$\overline{x_3}$	0	0	1	...	0	$\langle$ something $\rangle$
$\vdots$				$\vdots$		$\vdots$
x_ℓ	0	0	0	...	1	$\langle$ something $\rangle$
$\overline{x_\ell}$	0	0	0	...	1	$\langle$ something $\rangle$
	1	1	1	...	1	$\langle$ something $\rangle$

So two rows per variable.

Example: SUBSET-SUM (4)

Suppose that we want to choose a set of rows whose columns add up to the numbers on the bottom. Then, for any x_i, we would have to choose either the row headed by x_i or the one headed by $\overline{x_i}$. We wouldn't be able to choose both.

This is a bit like forcing every boolean variable x_i to be either true or false - the rows we'd choose above correspond to truth assignments!

We'll use the $\langle$ something $\rangle$s on the right to encode the clauses.

Example: SUBSET-SUM (5)

Suppose that c_1 is $(x_1 \vee \overline{x_2} \vee x_3)$. We can encode this in the first column on the right like so:

	1	2	3	...	ℓ	c_1	...
x_1	1	0	0	...	0	1	...
$\overline{x_1}$	1	0	0	...	0	0	...
x_2	0	1	0	...	0	0	...
$\overline{x_2}$	0	1	0	...	0	1	...
x_3	0	0	1	...	0	1	...
$\overline{x_3}$	0	0	1	...	0	0	...
$\vdots$				$\vdots$			$\vdots$
x_ℓ	0	0	0	...	1	0	...
$\overline{x_\ell}$	0	0	0	...	1	0	...
	1	1	1	...	1	?	...

That is, we've added a 1 in this column to every row corresponding to a literal in c_1.

Example 4: SUBSET-SUM (5)

We can repeat the same process for every clause:

	1	2	3	...	ℓ	c_1	c_2	...	c_k
x_1	1	0	0	...	0	1	0	...	0
$\overline{x_1}$	1	0	0	...	0	0	0	...	0
x_2	0	1	0	...	0	0	1	...	0
$\overline{x_2}$	0	1	0	...	0	1	0	...	0
x_3	0	0	1	...	0	1	0	...	0
$\overline{x_3}$	0	0	1	...	0	0	1	...	1
$\vdots$				$\vdots$				$\vdots$	
x_ℓ	0	0	0	...	1	0	0	...	1
$\overline{x_\ell}$	0	0	0	...	1	0	0	...	0
	1	1	1	...	1	?	?	...	?

Example 4: SUBSET-SUM (6)

Any satisfying truth assignment to ϕ gives us something between 1 and 3 in the second set of columns. Any non-satisfying assignment will set at least one clause from the second column to 0.

If we add a couple of dummy variables to every clause column, we can make them add up to 3 when there's a satisfying assignment:

	1	2	3	...	ℓ	c_1	c_2	...	c_k
$\vdots$				$\vdots$				$\vdots$	
d_1						1	0	...	0
d'_1						1	0	...	0
d_2						0	1	...	0
d'_2						0	1	...	0
$\vdots$								$\vdots$	
d_k						0	0	...	1
d'_k						0	0	...	1
t	1	1	1	...	1	3	3	...	3

Example 4: SUBSET-SUM (7)

Putting it all together:
To turn this into a SUBSET-SUM instance, just note that the numbers in the columns can never add up to more than 3 – so we can treat the rows as numbers in base 10 and never have to worry about one digit affecting another. So our S will just be the non-bottom rows in the table, and t will be the bottom row.

We can see that this construction will take polynomial time:

> The total table size is $(\ell + k)^2$, which is clearly polynomial in the size of ϕ. Each entry in the table is a 0, a 1, or a 3, and each value can be found with at worst a polynomial time lookup. The string value of t, $\langle t \rangle$ is $1^\ell 3^k$, which can also be found in polynomial time. So all told, this construction takes polynomial time.

And we've already given an overview of our justification as to why it works. So 3SAT $\leqslant_p$ SUBSET-SUM, and so SUBSET-SUM is $\mathcal{NP}$-complete.

Example 2: PARTITION-INTO-TRIANGLES

Definition of PARTITION-INTO-TRIANGLES:

Instance: A graph $G = (V, E)$.

Question: Can G be partitioned into disjoint triangles (i.e., can the vertices in V be split into pairwise disjoint sets of three such that each set is a trinagle in G)?

Certificate: The partitioning P
(we can treat this as a collection of triples of vertices in V).

Verifier: "On $\langle G = (V, E), P \rangle$:"

1. . Check that P lists each $v \in V$ exactly once in its triples, and that there are no extra vertices in P).
2. For all triples $\{a, b, c\} \in P$:
3. Check that $\{a, b\}$, $\{a, c\}$, and $\{b, c\}$ are all edges in E.
4. If all of these checks pass, *accept*, otherwise *reject*.

Check: is this polytime?

Why is PARTITION-INTO-TRIANGLES $\mathcal{NP}$-complete?

We reduce from 3SAT.

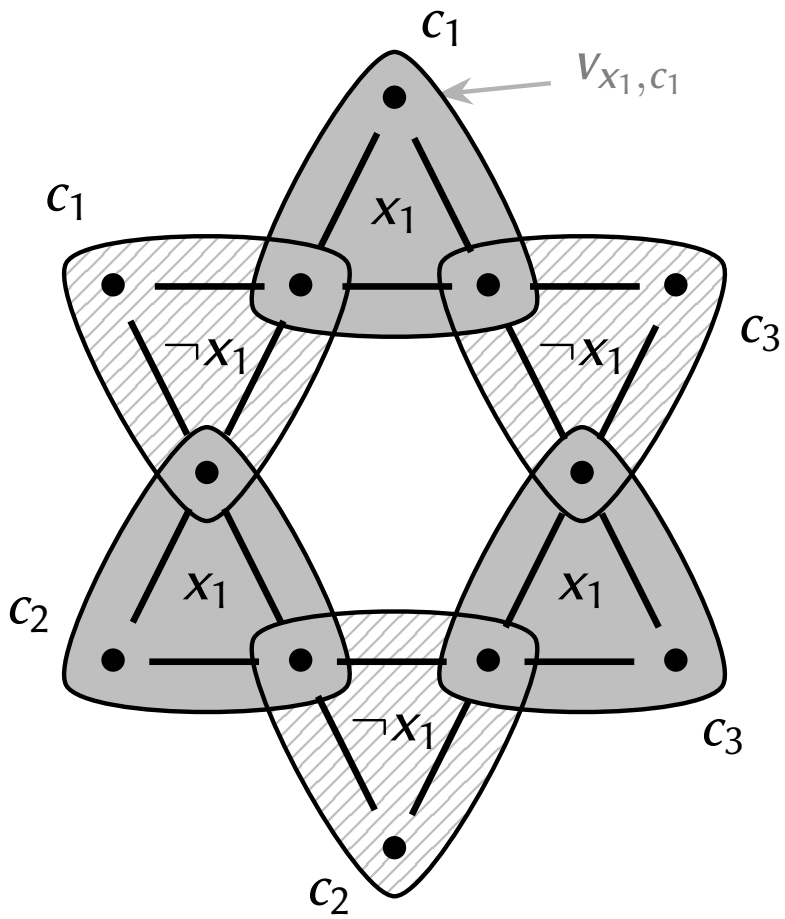
Suppose we are given an input 3CNF ϕ.

$$\text{e.g., } \phi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

- How can we program variable widgets into a graph G?
- How can we program the clauses?
- Will we need to do a bit extra to finish the job?

A Variable Widget

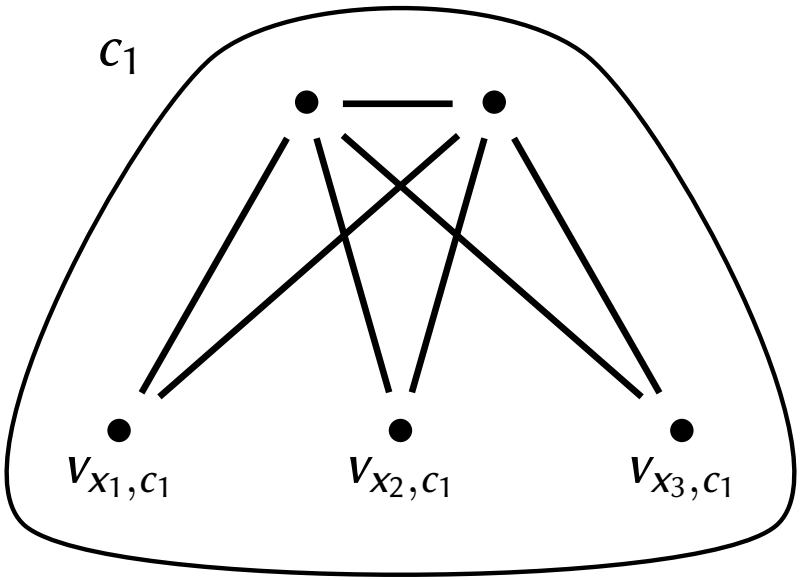
Here's a widget for x_1. Notice that it has two "leaves" for every clause C_j.



We'll connect the points of the "leaves" to the different clauses.

A Clause Widget

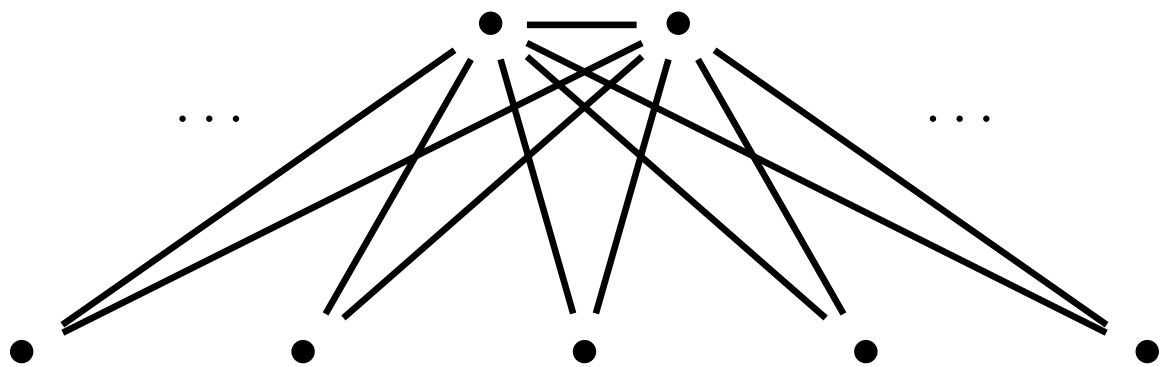
We can just attach the base of a triangle to each leaf corresponding to its literal:



We're almost there, but we've got a bunch of vertices left over. . .

A Garbage Collection Widget

It's basically like a large clause widget, but connected to every leaf in every variable widget:



We put in just enough of these to cover all the vertices (use a counting argument to see how many).

Is this reduction polytime?

Proof that $\phi \in$ 3SAT iff $\langle G \rangle \in$PARTITION-INTO-TRIANGLES

Suppose that $\langle \phi \rangle \in$ 3SAT, with n clauses and k variables:

- Then there exists some truth assignment that assigns at least one literal per clause to true.
- If we use that truth assignment to choose an assignment to triangles for the variable widgets, these literals will be free to be assigned with the n clause widgets.
- This will assign all of the inner nodes from the variable widgets to triangles, and will also assign all of the nodes from clause widgets to triangles. But it will leave $(k-1)n$ of the outer nodes from the variable widgets to be assigned.
- By assigning these nodes to the extra widgets from end of the construction.
- This will give us a decomposition of G into triangles.

  $\Rightarrow \langle G \rangle \in$ PARTITION-INTO-TRIANGLES.

Proof that $\phi \in$3SAT iff $\langle G \rangle \in$PARTITION-INTO-TRIANGLES (2)

Suppose that $\langle G \rangle \in$ PARTITION-INTO-TRIANGLES:

- Then There is some way of breaking G into disjoint triangles.
- In this partitioning, each of the clause widgets must contribute to exactly one triangle.
- The extra vertex in each of these triangles must correspond to some literal on the outside of the variable widgets.
- Since the variable widget enforces consistency in the variable assignments, the literals chosen by the clause widgets are consistent.
- $\Rightarrow$ These literal assignments can be extended into a consistent truth assignment.
- Since this truth assignment sets every clause to true, it is a satisfying assignment for ϕ.

  $\Rightarrow \langle \phi \rangle \in$ 3SAT.

3DM is $\mathcal{NP}$-Complete:

Recall the definition of 3DM:

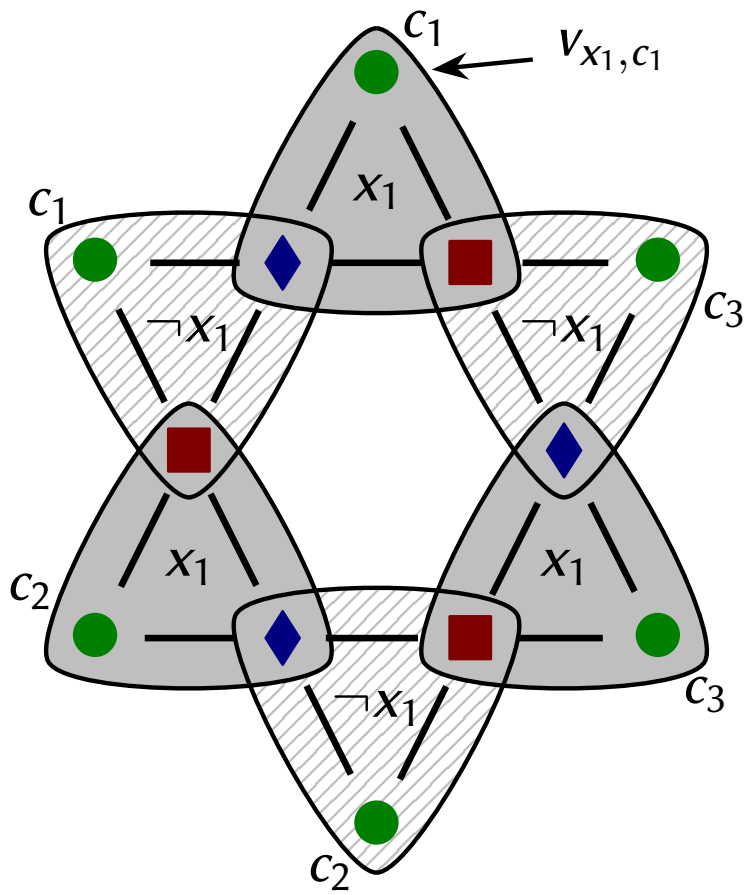
> **Instance:** Three sets A, B, and C, where $|A| = |B| = |C|$, and a set $T \subseteq A \times B \times C$.
>
> **Question:** Is there a subset $M \subseteq T$ such that:
>
> i) $|M| = |A|$,
>
> ii) For any distinct (a, b, c) and $(a', b', c') \in M$, $a \neq a'$, $b \neq b'$, and $c \neq c'$?

Idea: Re-use the reduction from 3SAT to PARTITION-INTO-TRIANGLES.

- We actually build the same graph G.
- We 3-colour G (3-colouring is usually hard, but not in this case).
- The sets A, B, and C are the *green*, *red*, and *blue* vertices.
- The set T is the set of triangles in G.

Colouring the Variable Widget

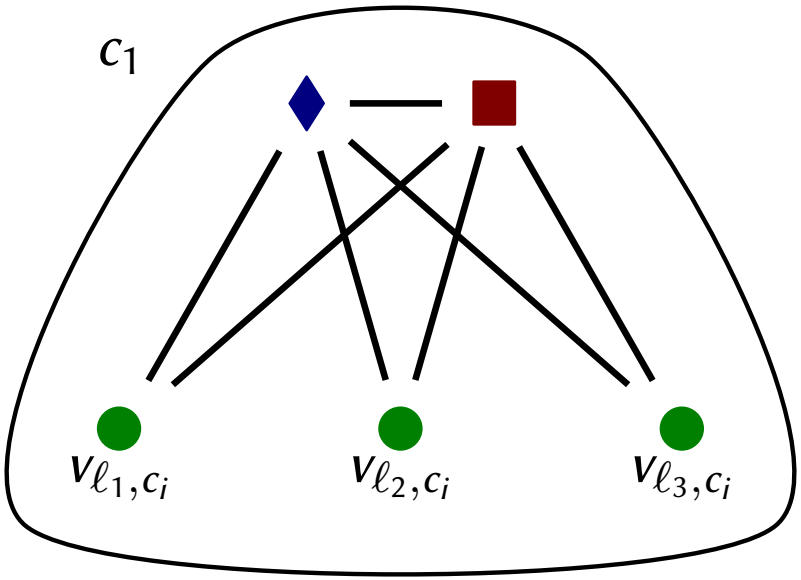
Here is one colouring for the variable widget:



Note: A = set of green circles, B = set of red squares, and C = set of blue diamonds.

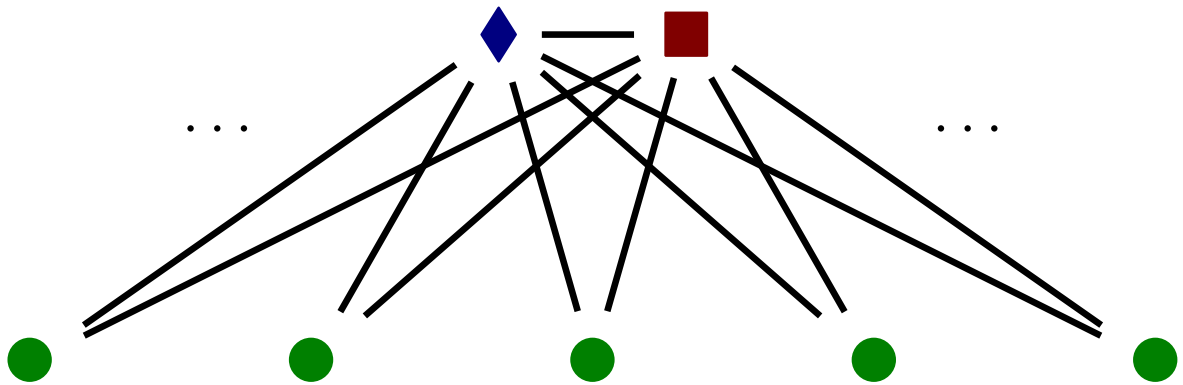
Colouring the Clause Widget

And we can extend this colouring to the clause widget:



Colouring the Garbage Collection Widget

And we can finish the colouring with the garbage collection widget:



If ϕ has n clauses and k vertices:

$$\begin{aligned} & \text{Number of green circles} && (|A|) \\ = \ & \text{number of red squares} && (|B|) \\ = \ & \text{number of blue diamonds} && (|C|) \\ = \ & 2nk. \end{aligned}$$

For the iff proof: See the PARTITION-INTO-TRIANGLES *reduction.*

Some Problems in $\mathcal{P}$

We've seen that a problem can be $\mathcal{NP}$-complete when the process of searching for a solution is difficult. But sometimes we can see problems that are very similar to $\mathcal{NP}$-complete problems, except the choices of solutions are limited enough that we can effectively search them. These include:

- 2SAT

  INPUT: A boolean formula ϕ in 2CNF.
  QUESTION: Does ϕ have a satisfying assignment?

- 2COL

  INPUT: A graph G.
  QUESTION: Is G 2-colourable?

Question: *Why might this be the case?*

2COL $\in \mathcal{P}$

High-Level Idea:

Given a graph $G = (V, E)$, we pick a starting point s and run a BFS of the graph. Every time we search a vertex, we colour it *red* if its distance from s is even, and *blue* if it is odd. Repeat this process for every component.

Once we're finished, just check to see if there are any neighbours of the same colour.

We can see that once we've chosen the colour of the first vertex, the possible colours of all vertices in its connected component are immediately fixed. So there's only one choice to worry about.

If we want to see if a graph G is 3-colourable, though, we have many choices to search through.

NP-Intermediate Problems

We have described the class $\mathcal{P}$ of efficiently solvable problems and the $\mathcal{NP}$-complete class of problems. Is every language in $\mathcal{NP}$ in one of these classes?

Ladner's Theorem: If $\mathcal{P} \neq \mathcal{NP}$, then there are problems in $\mathcal{NP}$ which are neither $\mathcal{NP}$-complete nor in $\mathcal{NP}$.

We call these the $\mathcal{NP}$-intermediate ($\mathcal{NPI}$) problems.

Question *If we don't know that $\mathcal{P} \neq \mathcal{NP}$, how do we know these languages exist?*

Answer: If $\mathcal{P} \neq \mathcal{NP}$, there's a 'gap' between the complexity of $\mathcal{P}$ and the $\mathcal{NP}$-complete problems, and there have to be languages in this gap.

$\mathcal{NP}$-Intermediate Problems (2)

Problem:

- We can't determine which languages are in this gap unless we know exactly where it is — and that would require us to know exactly how hard the $\mathcal{NP}$-complete problems really are.

Some suspected $\mathcal{NP}$-intermediate problems:

- Integer factorization.
- Graph Isomorphism.